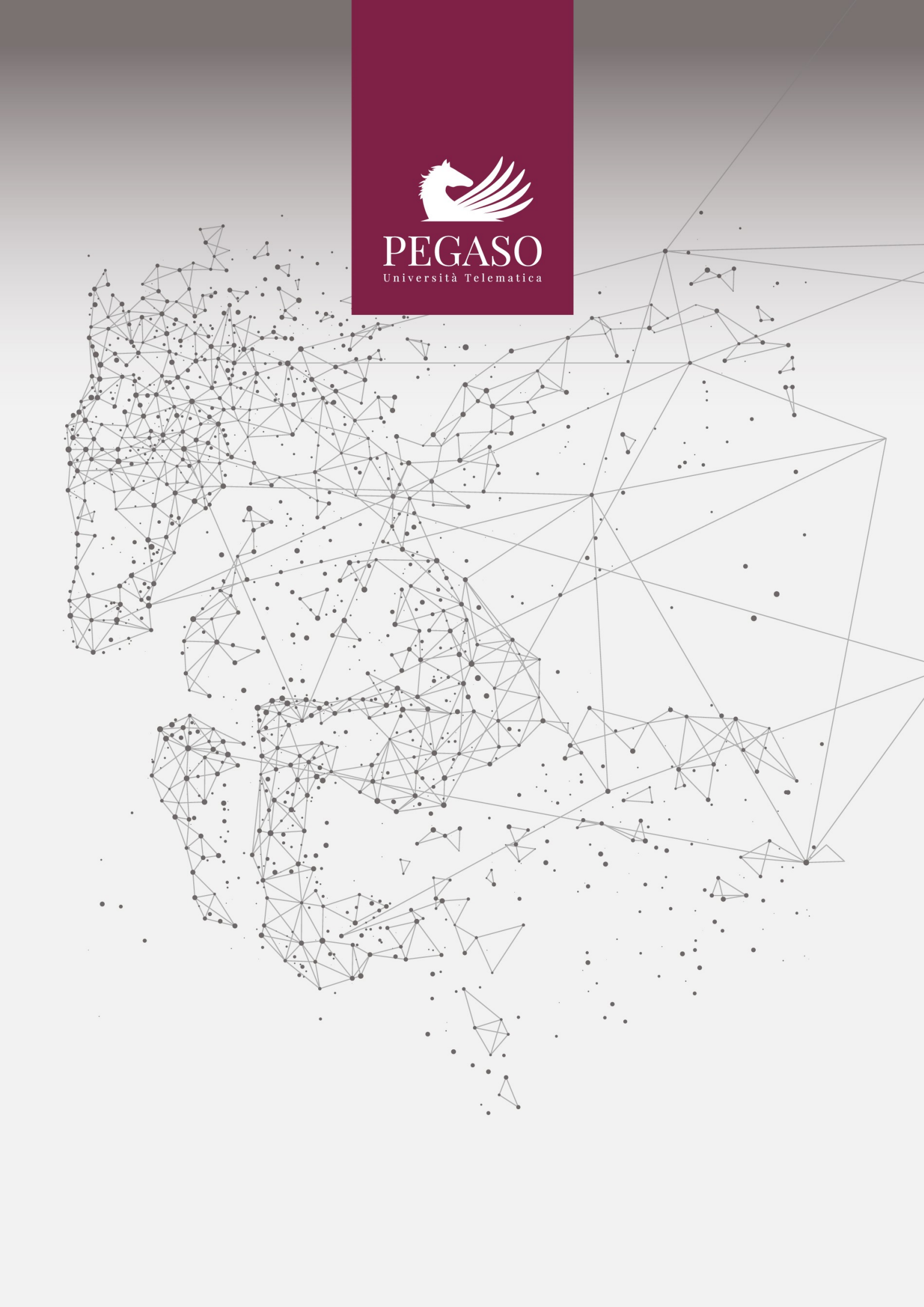

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTERAZIONI CON IL WEB 3
2. FORM INPUT.....6
3. REQBIN.....8
BIBLIOGRAFIA 11

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Interazioni con il web

Il web di oggi è molto spesso usato anche come mezzo per realizzare applicazioni che richiedono una interazione con l'utente che va oltre la normale navigazione. Un tipico esempio è la raccolta di informazioni dai visitatori del sito. Vediamo in dettaglio alcune parti essenziali.

In prima istanza scriviamo il codice necessario a far sì che si possa inviare una e-mail (per mezzo del client di posta normalmente presente sui nostri computer). Di solito questa tecnica molto semplice che non richiede nessun programma dal lato server, è usata per far inviare ad un indirizzo prefissato dei commenti o delle segnalazioni via e-mail. Come? Per mezzo degli elementi form e input.

L'esempio mostra direttamente la sintassi tipica usata con le form:

```
<form action=mailto:xxx.yyy@zzz.it method="post"
 enctype="text/plain">
<input type="text" name="subject" value="ciao" />
<input type="submit" name="empty" value="invia" />
</form>
```

L'attributo action dell'elemento form ha come valore una URI che determina l'azione della form stessa:

- nel caso lo schema della URI sia **mailto** viene aperto il client per l'invio di una e-mail all'indirizzo indicato;
- nel caso lo schema della URI sia **HTTP** al momento del submit i dati sono inviati al server e quel valore indica la URI di destinazione.

L'attributo **method** dell'elemento form corrisponde al metodo HTTP che sarà usato al momento dell'invio della form stessa per mezzo della fase di submit. Il suo valore indica in che modo le informazioni vengono inviate al server e può assumere due diversi valori:

- se il valore è **post** la trasmissione dei dati della form è costituita da due parti:
  - il browser (client) comunica al server che ci sarà una post all'indirizzo indicato nella action
  - in una trasmissione apposita il client trasmette al server i dati della form
- se il valore è **get** la trasmissione dei dati della form al server viene effettuata in un singolo passo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Da quanto detto si deduce che il metodo get è da preferire nel caso sia importante minimizzare il numero di trasmissioni. Comunque, il metodo get ha alcuni limiti che sono nelle dimensioni e di codifica dei parametri da trasmettere.

Inoltre, usando il metodo get i parametri sono immessi alla fine della URI ed è meno sicuro il suo uso rispetto alla post, anche se è solo una questione di facciata (la sicurezza si affronta passando su HTTPS).

Infine, il metodo post è indicato per le form complesse ed è necessario in quelle form che possono contenere file da trasmettere al server.

Quando il browser chiede una pagina web al server via HTTP, la richiesta che viene effettuata per mezzo di una get. Quindi l'uso di una form contenente il metodo get è equivalente del punto di vista del protocollo HTTP ad una normale richiesta di pagina (senza form).

L'attributo **enctype** dell'elemento form serve per indicare il formato di codifica da usare per trasmettere i dati della form verso il server. I possibili valori che può assumere:

- application/x-www-form-urlencoded è il valore predefinito. Questa tipo di codifica compone una stringa convertendo tutti i campi della form in una sequenza di associazioni nomecampo=valore
- text/plain è da usarsi con le form che hanno una action che ha schema mailto
- multipart/form-data è da usarsi soltanto con il metodo post. Questa codifica compone ogni campo della form in una o più parti di un documento di tipo MIME

Il **MIME** (Multipurpose Internet Mail Extensions) indica una estensione del formato delle e-mail per permettere la codifica di messaggi con caratteri diversi dall'ASCII, di messaggi non testuali, di messaggi con diverse tipologie di contenuto (immagini, testo, …).

Le RFC dalla 2045 alla 2049 sono il riferimento per il MIME, mentre nel protocollo HTTP è indicato il comportamento dei client e server con le post di tipo **multipart**.

Riassumendo:

- Il tag <form> è utilizzato per creare una form HTML ed acquisire degli input dall'utente
- L'elemento <form> può contenere uno o più elementi del tipo: <input>, <textarea>, <button>, <select>, <option> …

Alcuni degli attributi principali del tag <form> sono:

- **action**: contiene la **URL** a cui inviare i dati della form quando avviene il submit
- **method**: indica il tipo di metodo per l'invio
  - **get**
  - **post**

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **enctype**: solo per il metodo post, indica quale tipo di encoding dei dati occorre fare per l'invio al server:
  - **application/x-www-form-urlencoded** (predefinito)
  - **multipart/form-data**
  - **text/plain**
- **name**: specifica il nome della form
- **target**: specifica dove mostrare la risposta del server
  - **_blank**
  - **_self**
  - **_parent**
  - **_top**

L'interazione tipica di un utente con le pagine contenenti le form è la seguente:

- L'utente riempie gli elementi (input, …) nelle form sul browser (tipicamente le caselle di testo)
- L'utente seleziona nella form uno degli elementi input con type="submit", cioè fa click su un pulsante nella pagina
- I dati vengono inviati dal client al server, usando metodo e codifica indicati negli attributi della form dopo che il browser li ha codificati appositamente
- Il server elabora i dati inviati dal client e restituisce una risposta, che normalmente è una pagina HTML

Analizziamo dunque le tipologie di input che si possono avere all'interno di una form.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Form Input

L'elemento input si usa per definire gli elementi di controllo più diffusi nelle form. Gli attributi più comuni sono:

- **name**: indica il nome del campo, viene usato al momento dell'invio della form
- **value**: indica il valore predefinito del campo
- **type**: indica il tipo del campo

Di seguito i valori che può assumere il campo type:

- **text: casella di testo monoriga**
- **password**: come text ma il testo non è leggibile (****)
- **file**: controllo che consente di caricare un file
- **checkbox**: casella di spunta
- **radio**: radio button
- **submit**: bottone per trasmettere il contenuto del form
- **image**: bottone di submit sotto forma di immagine
- **reset**: bottone che riporta tutti i campi al valore iniziale
- **button**: bottone di azione
- **hidden**: campo nascosto

Affinché l'elemento input sia correttamente interpretato gli attributi type e name sono necessari.

Tutti gli input possono essere disabilitati utilizzando l'attributo disabled nella forma disabled = "disabled"

I tipi di input più comuni sono il text e il submit:

- Usando l'attributo type="text" in un elemento input si crea un campo che permette l'inserimento di una riga di testo.
- Usando type="submit" si crea un campo che, quando attivato, serve a inviare i dati della form.

Ecco un esempio di input text e submit:

```
<form action="XXX" method="POST">
<input type="text" name="testo" value="123" size="4" />
<input type="submit" name="empty" value="invia" />
</form>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Gli attributi per un input type text sono:

- name = text → nome del controllo
- value = text → eventuale valore iniziale
- size = n → lunghezza del campo (numero di caratteri)
- maxlength = n → massima lunghezza del testo (numero di caratteri)

Usando l'elemento input con il type="password" si ha la possibilità di creare un campo che permette l'inserimento di dati riservati. Questo non ha implicazioni con la sicurezza del trasferimento dei dati, che è comunque identica per tutti i campi della form. È invece solo una protezione sullo schermo che il browser può attuare, di solito sostituendo ogni carattere digitato con un simbolo *.

Con l'elemento input e il type="hidden" si chiede al browser che il campo in questione non sia presentato all'utente, ma che comunque sia inviato nella trasmissione al server al momento dell'invio della form. Con questo tipo generalmente si introducono dei valori predefiniti della form da inviare durante la fase di submit.

Con l'elemento input e il type="file" si crea un campo che permette la selezione di un file sul client che verrà trasmesso al server al momento dell'invio della form. In congiunzione con il type="file" si può usare l'attributo value per indicare il Content-Type dei file accettati.

Per i bottoni si utilizzano tre valori dell'attributo type:

- **submit**: per il bottone che provoca la spedizione del form al server
- **reset**: per il bottone che riporta il contenuto dei campi al valore originale
- **button**: per un generico bottone di azione

L'etichetta che compare nel bottone viene definita usando l'attributo **value**.

Da precisare che esiste anche il tag <button> che offre la possibilità di creare dei bottoni con un aspetto anche complesso: vi è infatti la possibilità di inserire il testo del bottone come contenuto del tag ed anche codice HTML all'interno del tag (testo formattato ma anche immagini).

```
<form action="http://aaa.bbb/ccc/dosomething" method="post">
<button type="button">Generico</button><br>
<button type="reset"><i>Azzera</i></button><br>
<button type="submit"><b>Invia</b></button>
</form>
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. ReqBin

Introduciamo il tool online ReqBin:

[Online API Testing Tool | Test Your API Online (reqbin.com)]

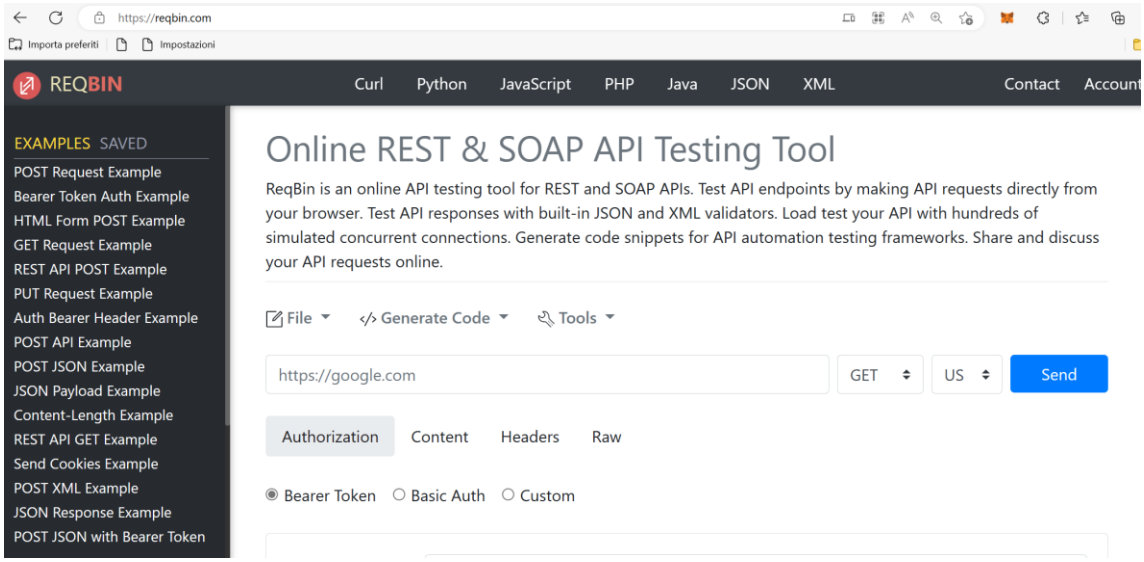


Figura 1 - Reqbin

Con Reqbin è possibile eseguire chiamate in GET / POST, generare script in python, Javascript ecc.

ATTENZIONE: le chiamate dall'interfaccia di ReqBin vanno a buon fine (ad es. richiamando http://www.google.com in GET) anche se inviate tramite uno script lato browser (da un dominio differente) se si usa un "**Bearer token**". Qualora si prelevasse lo script tramite "generate code / Javascript / Ajax" e si facesse un test da una propria pagina, non funzionerebbe proprio per via della mancanza di un Bearer token.

Reqbin mette a disposizione una serie di interfacce di "echo" che sono molto utili per poter eseguire dei test: ReqBin Echo

Di seguito la lista delle URL che è possibile richiamare:

- https://reqbin.com/echo/get/json
- https://reqbin.com/echo/post/json
- https://reqbin.com/echo/head/json
- https://reqbin.com/echo/get/xml
- https://reqbin.com/echo/post/xml
- https://reqbin.com/echo/post/form

Le response ottenute da queste interfacce, se non vi sono errori, saranno sempre del tipo "Successful" nel formato di risposta previsto.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ad esempio, se volessimo inviare una HTML POST dovremmo operare nel seguente modo:

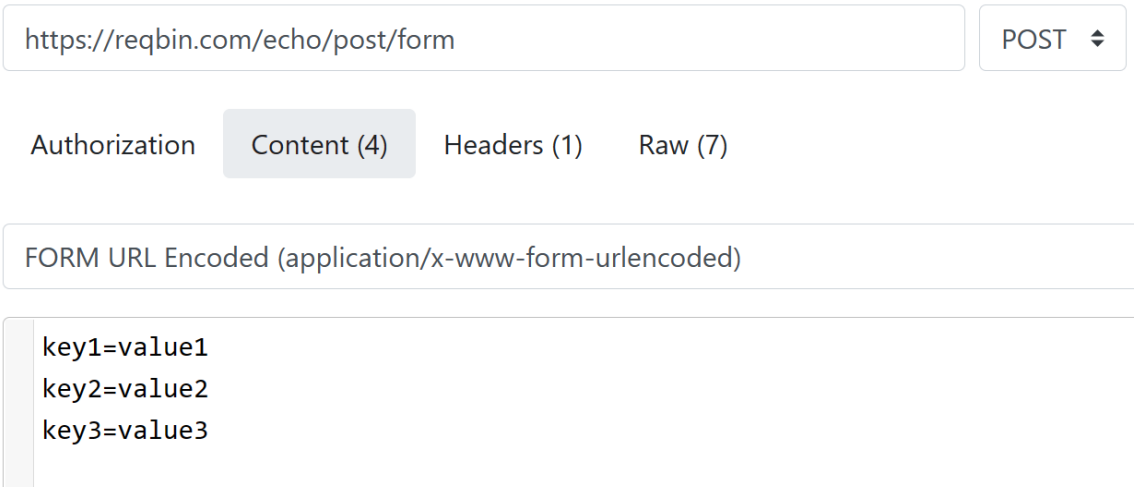


Figura 2 - HTTP POST - Content

(Accertarsi che l'Authorization sia sempre Bearer token)

Il corrispondente Raw è il seguente:

POST /echo/post/form HTTP/1.1

Host: reqbin.com

Accept: application/json

Content-Type: application/x-www-form-urlencoded

Content-Length: 37

key1=value1&key2=value2&key3=value3&=

La response sarà del tipo:

```
<html>

<body>
    <h1>Success</h1>
    <script>
        (function() {
            var js = "window['__CF$cv$params']={r
            var _0xh = document.createElement('if
            _0xh.height = 1;
```

Figura 3 - Successful HTTP POST

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Una richiesta POST in json è invece la seguente:

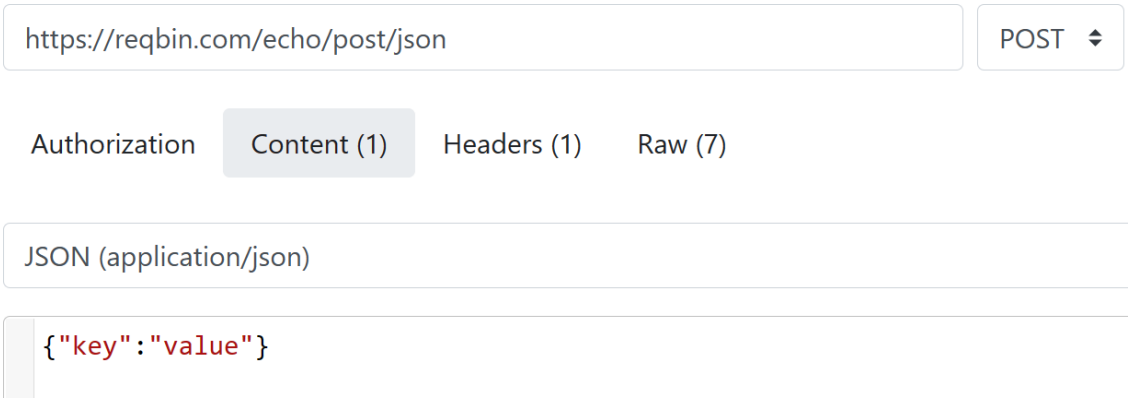


Figura 4 - HTTP POST in json

Il corrispondente Raw è il seguente:

POST /echo/post/json HTTP/1.1

Host: reqbin.com

Accept: */*

Content-Type: application/json

Content-Length: 15

{"key":"value"}

La response sarà del tipo:



Figura 5 - Successful HTTP POST

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010;
- S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).